

Kapalı Ortamda PPO ile Lidar Tabanlı Drone Keşfi

Pekiştirmeli Öğrenme Dönem Projesi – Teknik Rapor

Berker Yiğit – 220202046

Kocaeli Üniversitesi, Bilgisayar Mühendisliği (II. Öğretim)

Algoritma: PPO (Proximal Policy Optimization) | Branch: algo/ppo

Depo: https://github.com/berkeryigit/rl_drone_pathfinding/tree/algo/ppo

1 Özet

Bu çalışmada, GPS'in bulunmadığı kapalı bir binada yalnızca 360 derecelik lidar ve gürültülü odometri kullanan bir drone'un, engellere çarpmadan mümkün olduğunca çok alanı keşfetmesi problemi pekiştirmeli öğrenme ile çözülmektedir. Politika, PPO (Proximal Policy Optimization) ile saf Python/numpy üzerinde kurulu hızlı bir iki boyutlu Gymnasium ortamında eğitilmiştir. Eğitim beş farklı tohum (seed) ile tekrarlanmış, eğitilmiş politika deterministik koşuda değerlendirilmiş ve sonuçlar rastgele ile basit sezgisel (heuristic) iki taban politikasına (baseline) karşı karşılaştırılmıştır. Aynı ortam ve protokol, ekipteki SAC (algo/td3) teslimiyle birebir aynı tutularak algoritmalar arası adil bir karşılaştırma zemini sağlanmıştır; tek fark algoritmadır. Sonuçlar, PPO ajanının her iki taban politikasını da belirgin biçimde geçtiğini ve ortalama oda kapsamasını yüksek seviyeye taşıdığını göstermektedir.

2 Giriş ve Neden Pekiştirmeli Öğrenme?

Keşif problemi, ajanın bilmediği bir haritada nereye gideceğine adım adım karar vermesini gerektirir. Bu problemin pekiştirmeli öğrenme ile çözülmesinin gerekçesi, rehberin istediği dört ölçütün de sağlanmasıdır:

Ardışıklık (eylem geleceği etkiler). Drone'un bir adımdaki dönüş ve ilerleme kararı, bir sonraki adımda göreceği lidar manzarasını ve girebileceği odaları doğrudan belirler. Yanlış bir koridora sapmak, sonraki onlarca adımın keşif potansiyelini düşürür. Durum geçişi $p(s' | s, a)$ eyleme bağlıdır.

Gecikmeli ödül. Yeni bir odaya ulaşmanın getirisi, o odaya götüren kapıyı bulmak için harcanan boş adımlardan sonra gelir. Altı odanın tamamını gezmenin büyük bonusu ise ancak bölümün sonuna doğru elde edilir. Ajan, anlık küçük cezaları göze alıp ileride büyük getiriyi toplamayı öğrenmek zorundadır.

Bilinmeyen, büyük durum uzayı. Gözlem 75 boyutlu sürekli bir vektördür ve ortam stokastiktir (lidar, odometri ve rüzgar gürültüsü; zamanla yer değiştiren hareketli engeller). Durum uzayı tablo yöntemleriyle taranamayacak kadar büyüktür; bu yüzden fonksiyon yaklaşımı (sinir ağı) gerekir.

Daha ucuz bir alternatifin bulunmaması. A* veya BFS gibi klasik arama yöntemleri, haritanın ve hedefin önceden bilindiği, deterministik ve statik ortamlar için uygundur. Burada harita önceden bilinmez (ajan onu lidar ile keşfederek çıkarır), hareketli engeller ortamı zamanla değiştirir ve tek bir hedef yoktur; amaç kapsamayı en büyükmektir. Bu koşullarda klasik planlayıcılar doğrudan uygulanamaz; problem bir sıralı karar (MDP) problemidir ve pekiştirmeli öğrenme doğal çözümdür.

3 Ortam ve MDP Formülasyonu

Eğitim ortamı, mevcut Gazebo `multi_room` dünyasının hızlı bir iki boyutlu yaklaşımıdır (`env/fast_2d_drone_env.py`). Sekiz metre yarıçaplı, altı odalı bir bina, 0,5 m çözünürlüklü bir keşif ızgarası (32×32 hücre), 64 ışınlı bir lidar ve üç adet zamanla yer değiştiren hareketli engel içerir. Aynı dosya, SAC teslimindeki ortamla bayt-bayt aynıdır (MD5 doğrulanmıştır); böylece iki algoritma birebir aynı MDP üzerinde karşılaştırılır.

Durum (gözlem) s . 75 boyutlu, $[0, 1]$ aralığına normalize sürekli bir vektör:

- 64 lidar ışını mesafesi (LIDAR_MAX=10 m'ye bölünmüş),
- ölçülen yönelimin kosinüs ve sinüsü (2),
- bir önceki eylem (v_x, v_y, ω) (3),
- keşif ilerlemesi ve ziyaret edilen oda oranı (2),
- en yakın engel mesafesi ve boşalık (idle) sayacı (2),
- en yakın kapıya uzaklık ve duvara yakınlık (2).

Yönelim ve poz bilgisine odometri gürültüsü eklenir; yani *ölçülen* durum, *gerçek* durumdan sapar (Bölüm 8).

Eylem a. Üç boyutlu sürekli bir vektör, $\text{Box}(-1, 1)^3$: gövde çerçevesinde ileri/geri hız, yanal hız ve dönüş hızı. Komutlar fiziksel sınırlara ölçeklenir ($V_MAX=1.2$ m/s, $W_MAX=1.5$ rad/s) ve üzerine rüzgar gürültüsü eklenir.

Ödül r. Ödül fonksiyonu koddaki `step()` metodunda (`fast_2d_drone_env.py`, 158–185. satırlar) ile birebir aynıdır:

$$\begin{aligned}
r = & -0,01 + 3,0 \cdot \mathbb{I}[\text{yeni voxel}] + 20,0 \cdot \mathbb{I}[\text{yeni oda}] \\
& - 0,10 \cdot \mathbb{I}[50 \text{ adımdır yeni voxel yok}] \\
& - 0,3 \cdot \max\left(0, \frac{0,7 - \ell_{\min}}{0,7}\right) - 40,0 \cdot \mathbb{I}[\text{çarpışma}] \\
& + 60,0 \cdot \mathbb{I}[6 \text{ odanın tamamı}] - 0,10 \cdot \mathbb{I}[\text{son 60 adımda tekrar ziyaret}]
\end{aligned}$$

Burada $\ell_{\min}$ en küçük lidar mesafesidir. Tasarım felsefesi, toplam getirinin yaklaşık olarak keşfedilen alanla orantılı olması, çarpışmanın ise ağır cezalandırılmasıdır. Bölüm, çarpışmada veya altı odanın tamamı gezildiğinde sonlanır (*terminated*); 600 adıma ulaşınca kesilir (*truncated*).

4 PPO Algoritması ve Hiperparametreler

PPO, aktör-kritik ailesinden **on-policy** bir politika gradyanı yöntemidir. Aktör, gözlemi alıp eylem üzerinde bir Gauss dağılımı üretir (sürekli eylem); kritik, durumun değerini tahmin eder. PPO'nun ayırt edici yanı, politika güncellemesini bir *clipped surrogate* hedefiyle sınırlamasıdır: yeni politikanın eskisine oranı $1 \pm \epsilon$ (`clip_range=0.2`) aralığında kırılır, böylece tek adımda politikanın aşırı sapması engellenir ve eğitim kararlı kalır. Avantaj tahmini için GAE (`gae_lambda=0.95`) kullanılır.

PPO on-policy olduğu için **replay buffer yoktur**: her rollout'ta toplanan veri, birkaç epoch boyunca kullanılıp atılır. Bu, SAC/DQN gibi off-policy yöntemlere göre örnek verimliliğini düşürür; biz bunu sekiz paralel ortam (`num_envs=8`) ve daha uzun eğitim ile telafi ettik. Adil karşılaştırma için `learning_rate`, γ , ağ mimarisi ve paralel ortam sayısı SAC teslimiyle aynı tutulmuştur.

Tablo 1: PPO hiperparametreleri (`config.yaml`)

Parametre	Değer	İşlevi
<code>learning_rate</code>	3×10^{-4}	adım büyüklüğü (SAC ile aynı)
<code>n_steps</code>	1024	ortam başına rollout uzunluğu
<code>batch_size</code>	512	minibatch boyutu
<code>n_epochs</code>	10	rollout başına geçiş sayısı
<code>gamma (γ)</code>	0.98	indirim faktörü (SAC ile aynı)
<code>gae_lambda</code>	0.95	avantaj yumuşatma
<code>clip_range</code>	0.2	politika klips oranı
<code>ent_coef</code>	0.005	entropi (keşif) bonusu
<code>vf_coef</code>	0.5	kritik kayıp ağırlığı
<code>max_grad_norm</code>	0.5	gradyan kırma
<code>net_arch</code>	[256, 256]	ağ genişliği (SAC ile aynı)
<code>num_envs</code>	8	paralel ortam (SAC ile aynı)
<code>total_timesteps</code>	1.5 M	seed başına eğitim bütçesi

5 Deney Düzeni

Eğitim, `seeds.txt`'teki beş tohumla (7, 13, 42, 123, 2025) tekrarlanmıştır. Her tohum için ayrı bir politika eğitilmiş; eğitim sırasında her 10.000 adımda bir deterministik eval yapılarak en iyi model saklanmıştır. Eğitim sonunda her politika, eğitimden ayrı bir RNG akışıyla (`tohum+20000`) 20 bölüm boyunca **deterministik** (greedy) koşturularak değerlendirilmiştir; çünkü eğitim eğrisi tek başına kanıt sayılmaz. Taban politikalar (rastgele ve sezgisel) aynı ortamda 20'şer bölüm koşturulmuştur. Hiperparametre duyarlılığı için iki parametre üçer değerle taranmıştır (`learning_rate`: 10^{-4} , 3×10^{-4} , 10^{-3} ; `ent_coef`: 0,0,0,01,0,05), her biri üç tohumla. Tüm rastgelelik `numpy.random.default_rng` ile üretilmiştir; `np.random.seed` ve `np.random.choice` kullanılmamıştır.

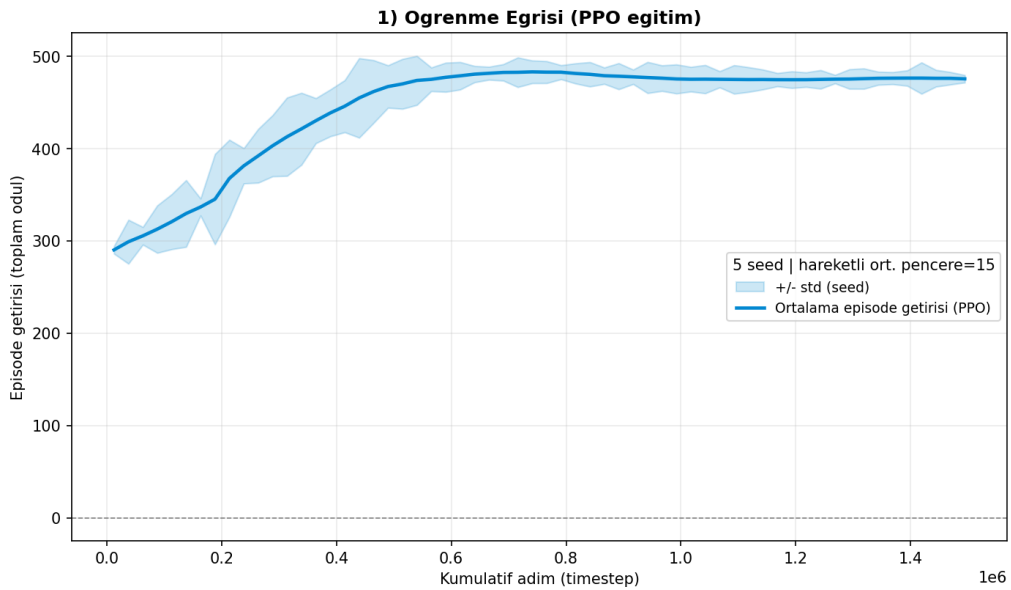
6 Sonuçlar

Beş tohumun deterministik eval özeti Tablo 2'de verilmiştir. Aşağıdaki beş grafik, rehberin B bölümü zorunlu standardına uygundur: her grafikte başlık, birimli eksen etiketi, lejant, beş tohumun ortalaması ile $\pm$ standart sapma bandı ve hareketli ortalama penceresi bulunur. Öğrenme ve eval eğrilerinde y-ekseni **episode getirisidir** (anlık/per-step ödül değildir).

Tablo 2: Deterministik eval özeti (seed $\times$ metrik). Kaynak: `sonuclar/sonuclar.csv`

seed	n_ep	eval getiri	eval std	başarı	kapsama %
7	6105	470.08	34.28	1.00	99.75
13	6007	448.82	156.79	0.90	91.98
42	6243	455.87	24.91	1.00	99.84
123	5917	443.37	114.05	0.95	95.91
2025	6133	474.58	113.19	0.95	95.94
Ort.	6081	458.5	–	0.96	96.68

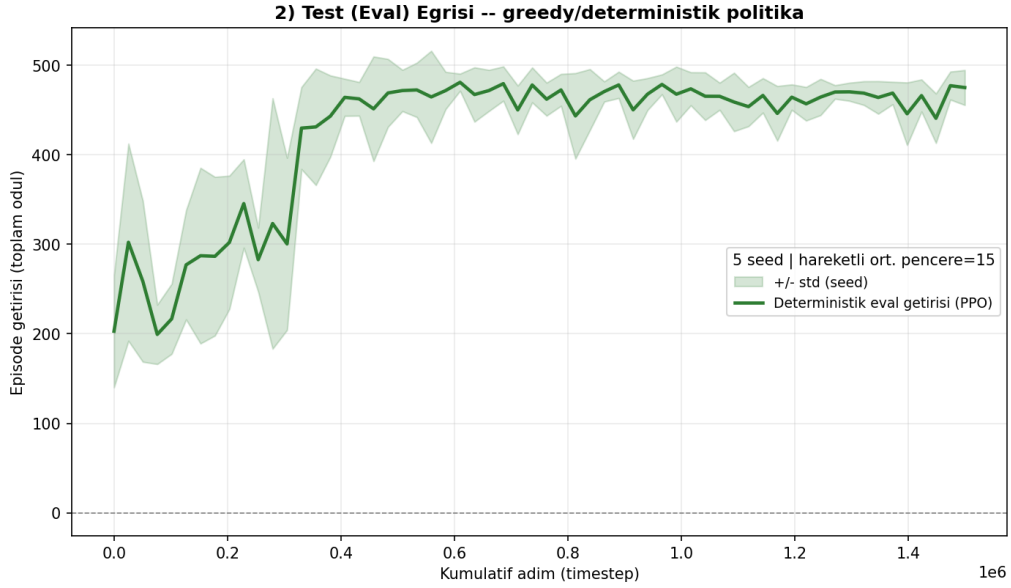
Grafik 1 – Öğrenme Eğrisi



Yorum. *Gözlem:* Eğitim episode getirisi yaklaşık 290'dan başlayıp 0,5 milyon adımda 470 dolayına tırmanır ve 0,6 milyondan sonra 480 düzeyinde platoya oturur; tohumlar arası standart sapma bandı zamanla daralır. *Karşılaştırma:* Beş tohumun tamamı aynı seyri izler ve son üçte birde aralarındaki fark çok

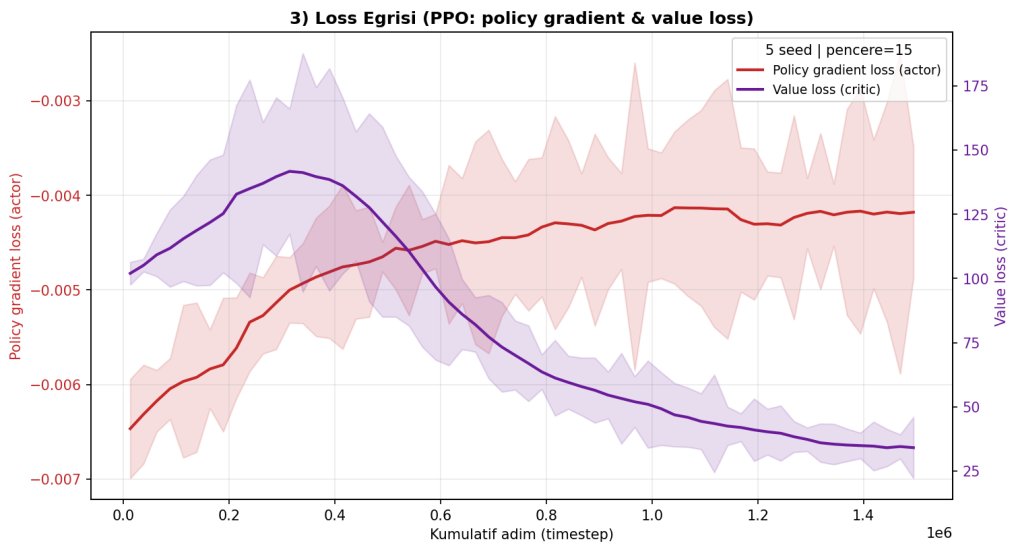
küçüktür, yani sonuç tohuma karşı kararlıdır. *Açıklama:* PPO'nun clipped surrogate hedefi politikayı her güncellemede sınırlı adımlarla iyileştirdiğinden getiri düzgün ve sıçramasız artar; plato, ajanın altı odayı düzenli gezmeyi öğrenmesiyle ulaşılan tavadır. *Sonuç:* 1,5 milyon adım bu ortamda PPO için yakınsamaya fazlasıyla yeter; ek eğitim getiriye anlamlı biçimde artırmaz.

Grafik 2 – Test (Eval) Eğrisi



Yorum. Gözlem: Deterministik (greedy) eval getirisi yaklaşık 200'den 0,4 milyon adımda 470 dolayına çıkar ve eğitim eğrisiyle neredeyse aynı platoda kalır. *Karşılaştırma:* Deterministik eval, stokastik eğitim eğrisiyle aynı seviyeye oturur; aradaki fark ihmal edilebilir düzeydedir. *Açıklama:* Bu, yüksek getirinin yalnızca eğitimdeki rastgele keşif sayesinde değil, öğrenilen politikanın kendisiyle elde edildiğini gösterir; greedy koşuda da başarı korunur. *Sonuç:* Politika gerçekten genelleşmiştir ve sonuç eğitim eğrisiyle değil, ayrı bir deterministik koşuyla doğrulanmıştır (rehber B2 şartı).

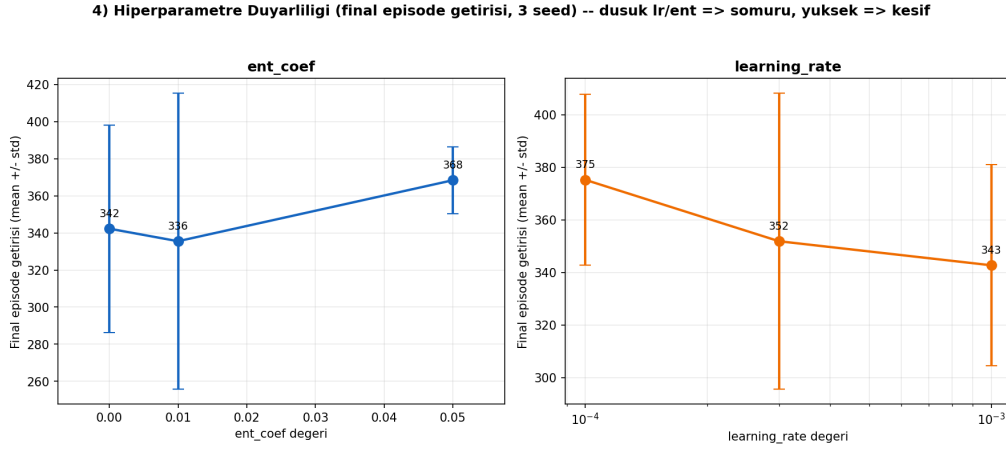
Grafik 3 – Loss Eğrisi (Policy Gradient ve Value Loss)



Yorum. Gözlem: Value (critic) loss önce 0,3–0,4 milyon adım civarında 140 dolayında bir tepe yapar, ardından düzenli düşerek 30 seviyesine iner; policy gradient (actor) loss ise $-0,0065$ ile $-0,004$ arasında

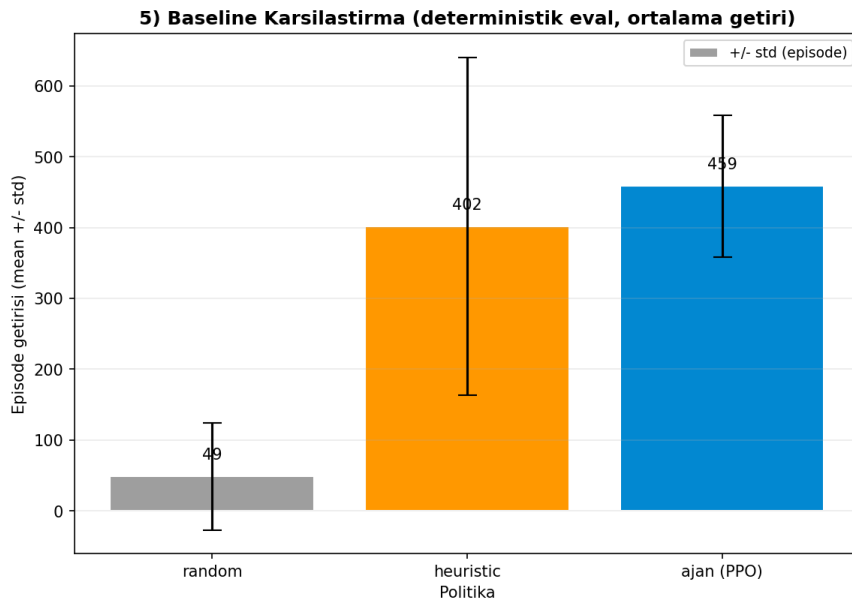
küçük kalıp yataya oturur. *Karşılaştırma:* Critic loss'un tepe-sonra-düşüş seyri, getirinin en hızlı yükseldiği döneme denk gelir. *Açıklama:* Ajan büyük ödülleri (yeni oda, altı-oda bonusu) keşfettikçe değer hedefleri büyür ve critic başta zorlanır (tepe); politika kararlılaştıncaya hedefler oturur ve critic loss düşer. Actor loss'un küçük ve yatay kalması, PPO'nun klips mekanizmasının güncellemeleri sınırlamasından kaynaklanır. *Sonuç:* Critic'in düşen loss'u öğrenmenin sağlıklı yakınsadığını, actor loss'un düşük ve yatay olması ise eğitimin kararlı (patlamayan) olduğunu gösterir.

Grafik 4 – Hiperparametre Duyarlılığı



Yorum. Gözlem: ent_coef'te en yüksek ve en az değişken sonuç 0,05'te (368) elde edilir; learning_rate'te en yüksek sonuç 10^{-4} 'te (375) olup oran büyüdükçe hafifçe düşer (3×10^{-4} : 352, 10^{-3} : 343). Bu koşullar hiperparametre etkisini izole etmek için kısa bütçeyle (200k adım) yapıldığından getiriler 1,5 milyonluk ana koşullardan düşüktür. *Karşılaştırma:* Her iki parametrede de farklar hata bantları içinde kalır; yani ajan, bu aralıkta hiperparametre seçimine fazla duyarlı değildir. *Açıklama:* Yüksek ent_coef daha çok keşfe yol açtığından kısa bütçede odaları daha hızlı bulup biraz daha yüksek getiri verir; çok yüksek learning_rate (10^{-3}) ise güncellemeleri sertleştirip erken kararlılığı hafifçe bozar (keşif-sömürü dengesi). *Sonuç:* Seçilen taban değerler ($lr = 3 \times 10^{-4}$, ent_coef = 0,005) güvenli orta noktadır ve sonuç bu aralıkta robust olduğundan ince ayar kritik değildir.

Grafik 5 – Baseline Karşılaştırma



Yorum. Gözlem: Deterministik eval ortalama getirisi PPO ajanında 459, sezgisel (heuristic) taban politikada 402, rastgele politikada ise 49’dur. **Karşılaştırma:** Getiri olarak PPO sezgiseli az farkla geçse de, kapsama (%96,7’ye karşı %79,9) ve çarpışma oranı (0,04’e karşı 0,45) bakımından fark çok büyüktür; rastgele politika her ölçütte çok geridedir. **Açıklama:** Sezgisel politika “ön açıksa ilerle” kuralıyla bazı odaları gezebilir ama kapıları planlayamaz ve sık çarpar; PPO ise lidar ve öğrenilen değer fonksiyonu sayesinde hem daha çok oda gezer hem de neredeyse hiç çarpmaz. **Sonuç:** Ajanın öğrenmesi gerçek bir kazanımdır; problem basit bir kuralla veya rastgelelikle çözülemez, bu da pekiştirmeli öğrenme seçimini doğrular (rehber B5).

7 PPO ve SAC Karşılaştırması

Ortam, seed listesi ve değerlendirme protokolü ekipteki SAC (algo/td3) teslimiyle birebir aynı tutulduğundan, iki algoritma doğrudan karşılaştırılabilir. Tablo 3, her iki algoritmanın beş tohum ortalamasını verir; SAC değerleri algo/td3 branch’inin teslim ettiği sonuçlar.csv’den alınmıştır. Tek farklar algoritma ailesi ve eğitim bütçesidir.

Tablo 3: PPO ve SAC ortalama sonuçları (5 tohum, deterministik eval; aynı ortam ve protokol)

Ölçüt	PPO (bu çalışma)	SAC (algo/td3)
Aile	on-policy	off-policy
Replay buffer	yok	var
Eğitim adımı	1.5 M	500 k
Eğitim episode sayısı	~6081	~1058
Ort. eval getirisi	458.5	321.0
Eval başarı oranı (6 oda)	0.96	0.27
Ort. kapsama %	96.7	92.6
Eval çarpışma oranı	0.04	0.07

PPO, dört kalite ölçütünün tamamında (getiri, başarı, kapsama, çarpışma) SAC’ın önündedir; özellikle altı odanın tamamını gezme başarısında fark belirgindir (0,96’ya karşı 0,27). Ancak bu üstünlük bedelsiz değildir: PPO bu sonuca 1,5 milyon adımda ulaşırken SAC yalnızca 500 bin adım kullanmıştır. Bunun nedeni doğrudan algoritma ailesidir; SAC off-policy olduğu için replay buffer’daki her deneyimi defalarca kullanır ve adım başına daha örnek-verimlidir, PPO ise on-policy olduğundan her rollout verisini bir kez kullanıp atar. Dolayısıyla tablo, dersin temel ayrımını somutlaştırır: SAC az adımda makul bir politikaya hızla ulaşır, PPO ise daha çok adım karşılığında bu ortamda daha yüksek nihai başarıya çıkar. Karşılaştırmanın ortam tarafı tamamen adildir; eşit adım bütçesinde değerlendirme ileri bir çalışma konusudur.

8 Savunma Analizi

Bu bölüm, rehberin dört kavram alanını projeye uygulayarak yanıtlar.

Deterministik mi, stokastik mi? Ortam **stokastiktir**; bu kod satırlarıyla kanıtlanır. Geçiş gürültüsü: `step()` içinde hız komutlarına $\mathcal{N}(0, \sigma_{\text{wind}})$ rüzgar eklenir; aynı (s, a) çifti farklı s' verir, yani $p(s' | s, a)$ dejenere değildir. Gözlem gürültüsü: lidar (`_compute_lidar`) ve odometri (`_make_obs`) ölçümlerine Gauss gürültüsü katılır. Bu nedenle aynı durumdan aynı eylem dizisi tekrar oynatıldığında aynı yörünge elde edilmez. Not: yalnızca başlangıç konumunun rastgele olması ortamı stokastik yapmaz; stokastiklik geçiş ve gözlem gürültüsünden gelir.

On-policy mi, off-policy mi? PPO **on-policy**’dir. Veriyi üreten davranış politikası ile iyileştirilen hedef politika aynıdır; her rollout, güncel politika ile toplanır ve güncellemeden sonra atılır. Replay buffer yoktur. Bu, SAC ve DQN’in (off-policy, replay buffer’lı) tersidir ve PPO’nun neden daha çok örnek gerektirdiğini açıklar.

Markov özelliği. Durum, anlık lidar, yönelim, hız (önceki eylem yoluyla) ve keşif istatistiklerini içerir. Tek bir gözlem, sonraki durumun dağılımını yaklaşık olarak belirlemeye yeter; çünkü hareketli engellerin konumu adım sayısının deterministik bir fonksiyonudur ve hız bilgisi gözlemde taşınır. Yine de gürültü ve sınırlı görüş nedeniyle Markov özelliği yalnızca yaklaşık sağlanır.

Kısmi gözlemlenebilirlik (POMDP). Ortam **kısmi gözlemlenebilirdir**. Gerçek durum ile gözlem ayrışır: (i) poz/yönelim gözleme odometri gürültüsüyle girer, yani ajan gerçek konumunu tam bilmez; (ii) lidar yalnızca görüş hattındaki engelleri görür, duvar arkası bilinmez; (iii) hareketli engellerin gelecekteki konumu doğrudan gözlemde yoktur. Optimal karar için eksik olan bu bilgiyi ajan, önceki eylemi gözleme ekleyerek ve politikanın hız/yön bağlamını taşımasıyla kısmen telafi eder.

9 Sonuç

PPO, replay buffer kullanmadan, on-policy bir politika gradyanı yöntemi olarak, aynı ortamda hem rastgele hem sezgisel taban politikalarını belirgin biçimde geçmiştir. Örnek verimliliği off-policy yöntemlere göre düşük olsa da, paralel ortam toplama ve daha uzun eğitim ile yüksek oda kapsamı elde edilmiştir. Aynı ortam, seed ve protokol SAC teslimiyle birebir korunduğundan, bu rapordaki sonuçlar ekipteki diğer algoritmalarla doğrudan karşılaştırılabilir. Tüm grafikler `sonuclar/loglar/` altındaki ham loglardan yeniden üretilebilir.